

Trivia KATA

CONTENTS

목차

CHAPTER 1

Trivia KATA 소개

CHAPTER 2

리팩토링을 위한 Unit Test 준비

CHAPTER 3

리팩토링 시작

Chapter1

Trivia KATA 소개

Trivia Game 이란?

- Trivia Game : 미국과 영국 등 영어권 나라의 Pub 에서 즐기는 “**상식퀴즈**” 게임
- (모두의 마블 같은) 보드판이 있고, 각 칸에는 퀴즈의 주제가 써 있음.
- 퀴즈의 주제 : **팝, 락, 과학, 스포츠**
- 주사위를 굴러 말을 이동시키고, 말이 도착한 지점에 대한 주제의 퀴즈를 맞추어야 한다.



모두의 마블 이미지

이미지출처 : 나무위키, KAKAO 모두의 마블 게임 소개 페이지

(<https://namu.wiki/w/%EB%AA%A8%EB%91%90%EC%9D%98%EB%A7%88%EB%B8%94%20for%20kakao/%EB%A7%B5>)

Legacy 코드 다운로드

사외링크

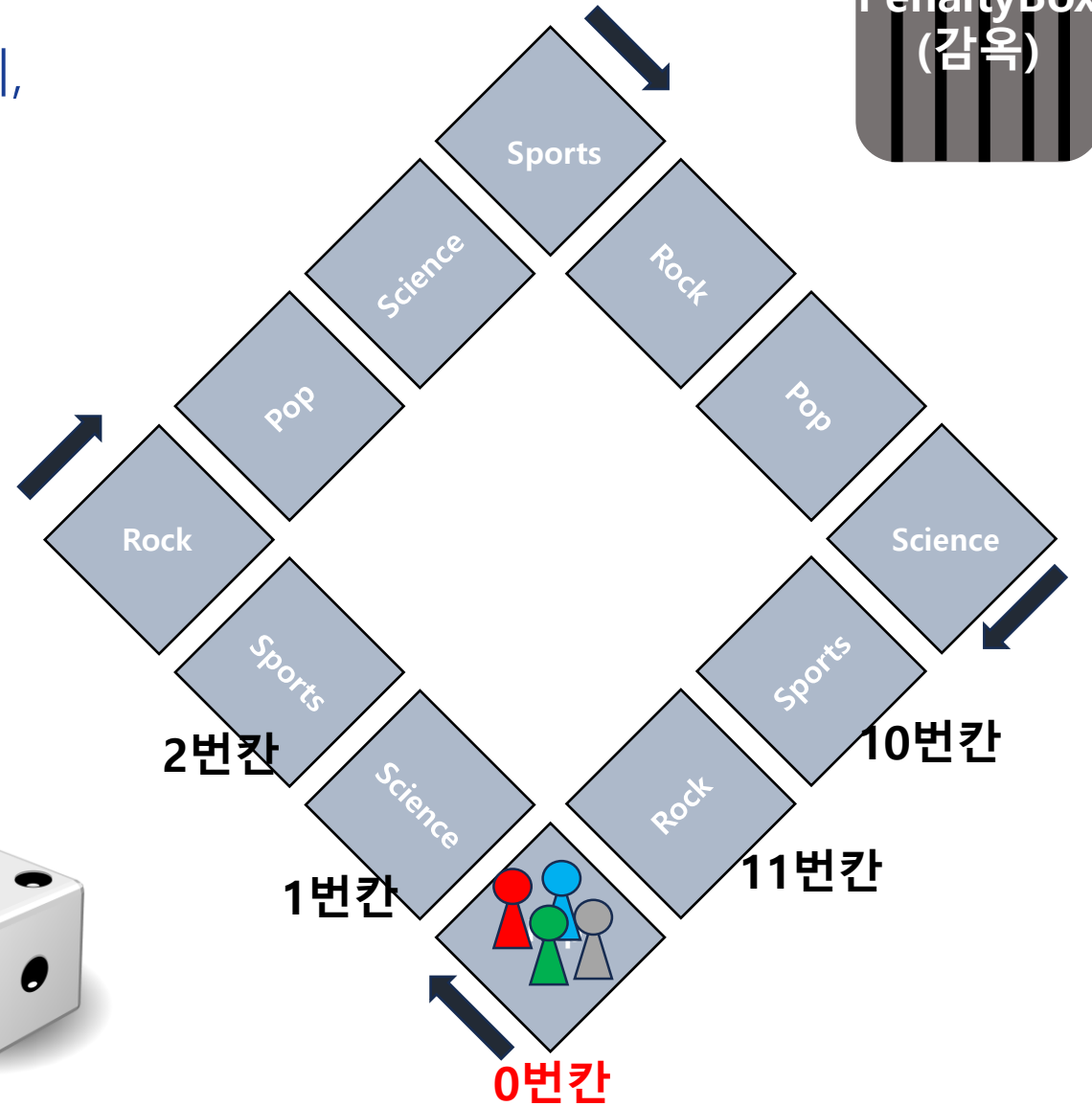
- <https://github.com/mincoding1/Trivia>

Trivia Game 예시 1

0 ~ 11번 칸 까지, 총 12개 칸으로 이뤄진 맵 위에,
총 4명의 플레이어가 게임을 한다고
가정한다.

모두 0번에서 출발한다.

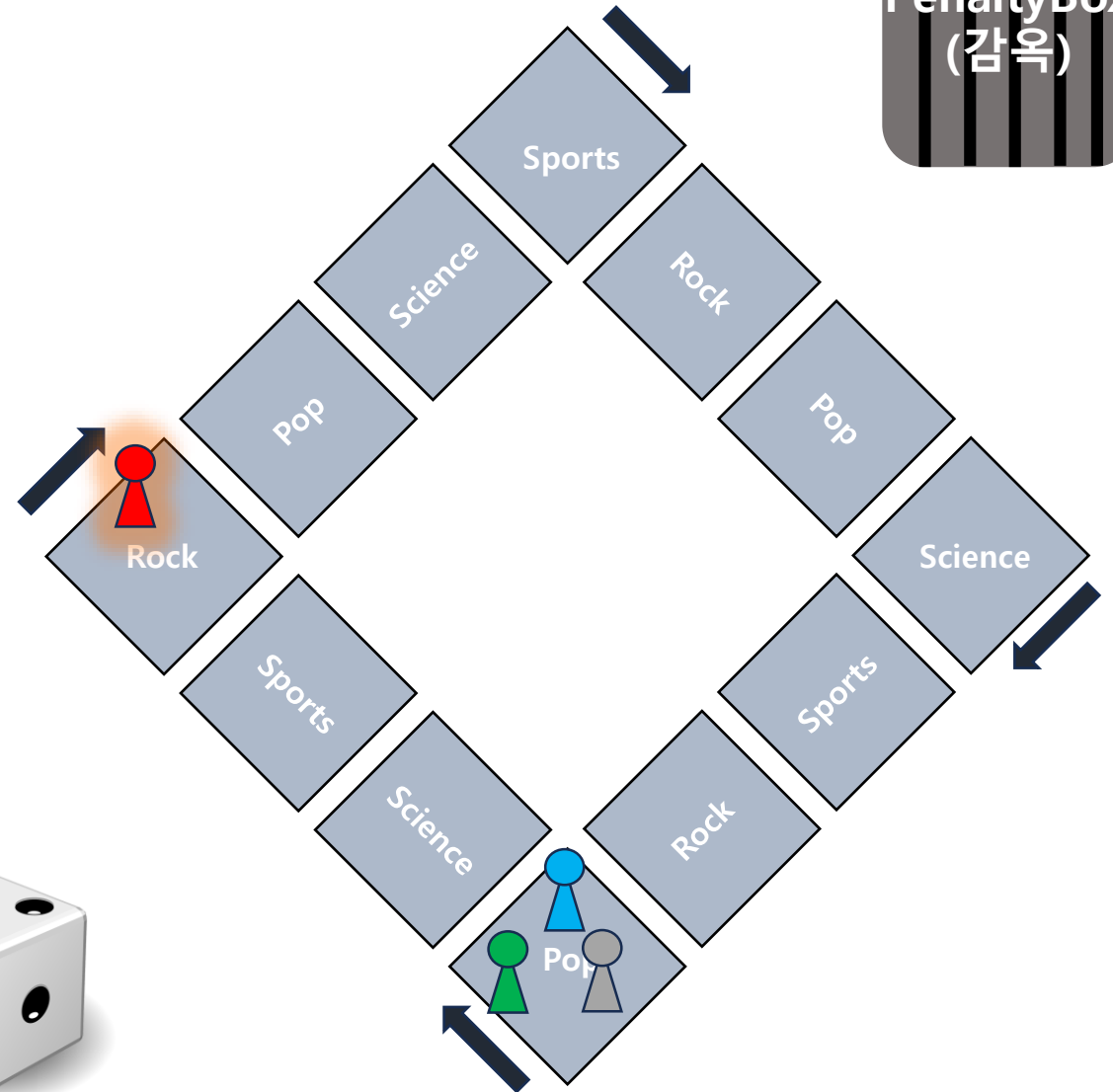
플레이어는 한 명씩 주사위를 굴러,
주사위 눈금만큼 말을 이동시킨다.



Trivia Game 예시 2

첫 번째 Player이
눈금 3이 나왔다면 세 칸 이동한다.

3번 칸의 주제는 Rock이다.
Rock 음악에 대한 상식 퀴즈를 맞춰야 한다.



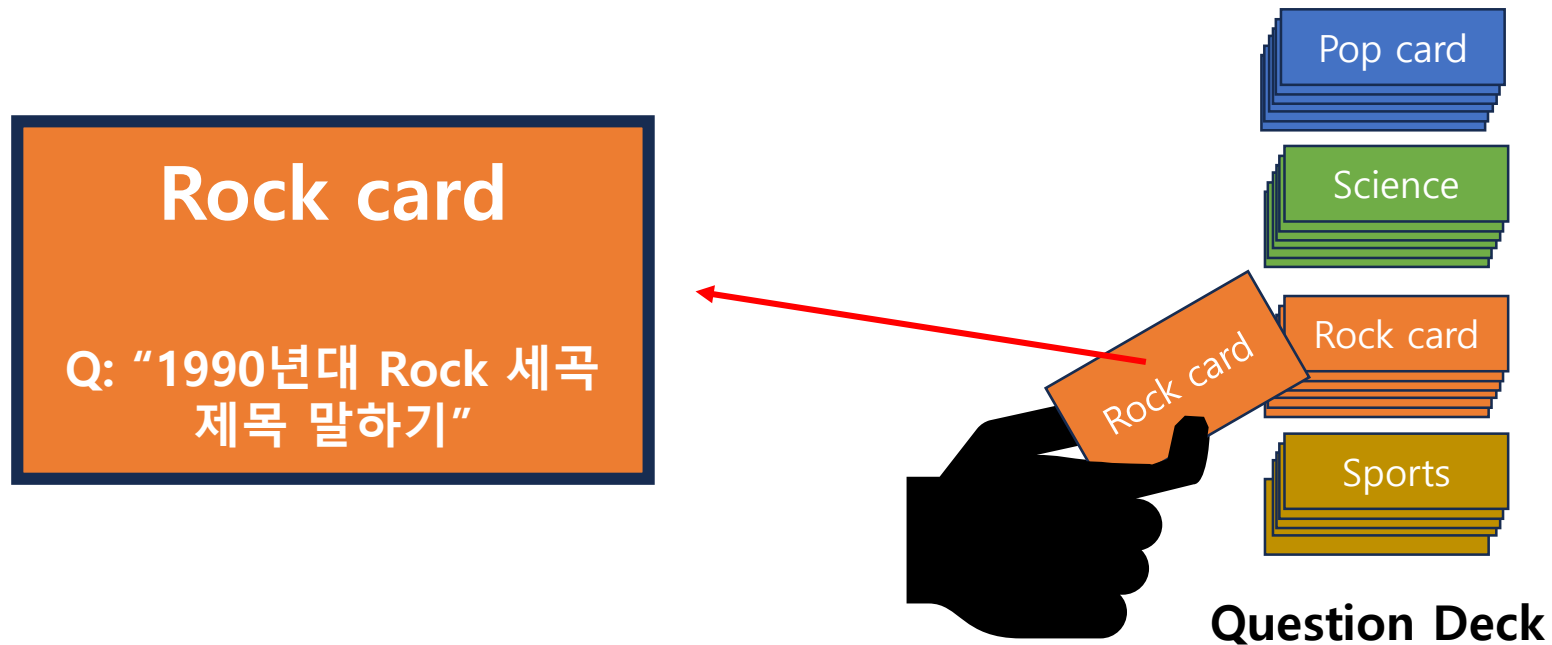
Trivia Game 예시 3

Rock 에 도착한 플레이어는

Question Deck 에 쌓여 있는 Rock 카드를 한 장 뽑는다.

이 카드에는 퀴즈 내용이 적혀 있다.

퀴즈를 맞추면 Coin 1개를 얻고, 틀리면 감옥으로 가야한다.



Trivia Game 예시 4

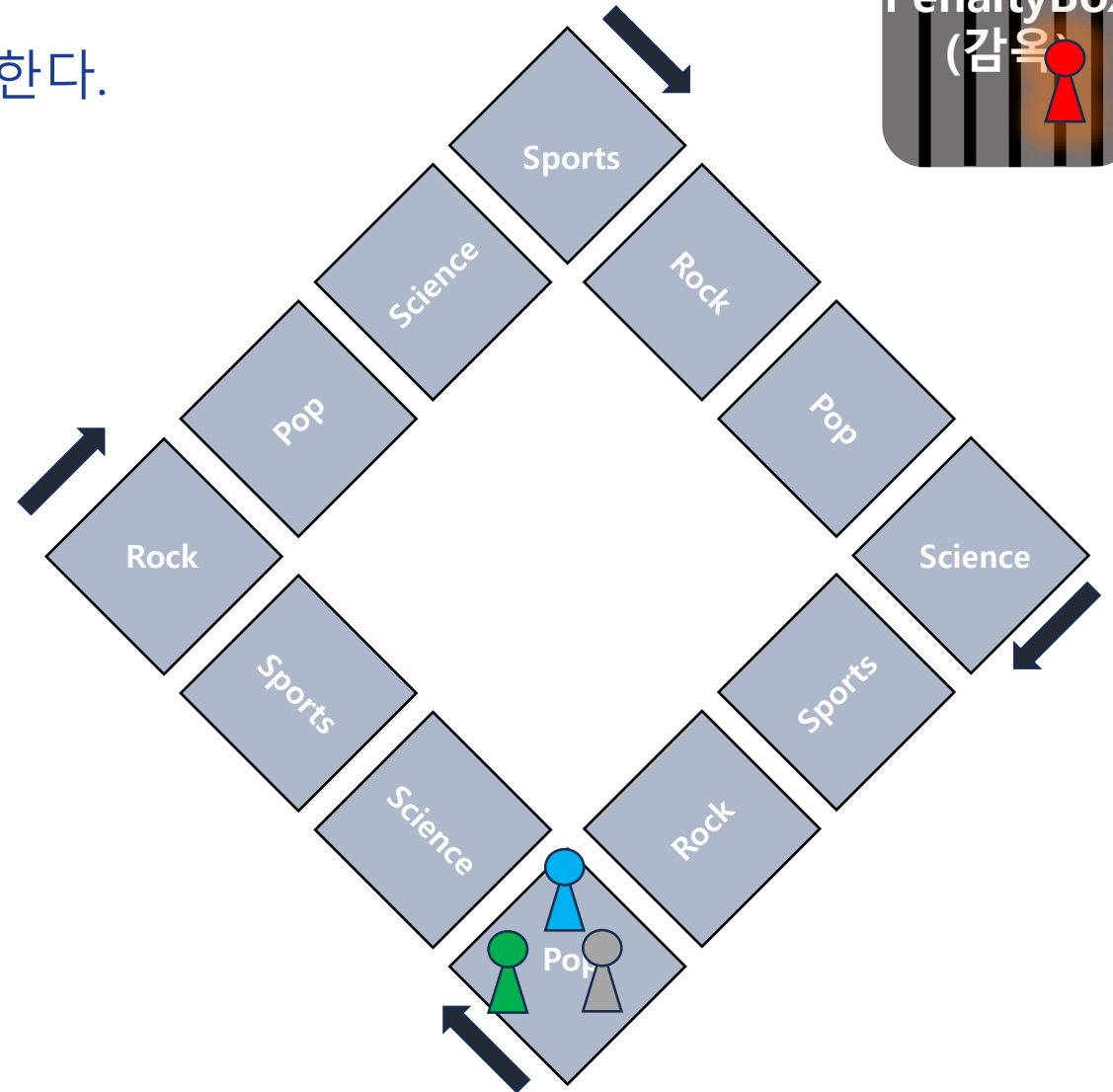


플레이어가 퀴즈에 정답을 맞추지 못했다고 가정한다.
따라서 감옥으로 말을 옮긴다.

돌아오는 턴에서
주사위의 눈금이 홀수일때만
감옥 탈출과 동시에 말을 이동시킬 수 있다.

위와 같은 방법으로 게임이 반복된다.

**총 6개의 코인을 먼저 얻는 사람이,
이 게임의 승자가 된다.**



Trivia Game 세부규칙1

게임을 시작할 때, 플레이어를 추가한다.

플레이어의 초기상태는 위치는 0, 코인은 0 이다.

```
public boolean add(String playerName) {  
    players.add(playerName);  
    places[howManyPlayers()] = 0;  
    purses[howManyPlayers()] = 0;  
    inPenaltyBox[howManyPlayers()] = false;  
  
    System.out.println(playerName + " was added");  
    System.out.println("They are player number " + players.size());  
    return true;  
}
```

기존 코드의 Game.add 메서드, place는 0 (시작지점) Purses 는 0 (코인) 으로 확인할 수 있다.

Trivia Game 세부규칙2

감옥은 돌아오는 턴에서 홀수의 주사위 눈금이 나와야 탈출할 수 있다.
탈출 직후, 해당 주사위 눈금만큼 이동한다.

```
public void rolling() {
    Random rand = new Random();
    int roll = rand.nextInt(6) + 1;

    System.out.println(players.get(currentPlayer) + " is the current player");
    System.out.println("They have rolled a " + roll);

    if (inPenaltyBox[currentPlayer]) {
        if (roll % 2 != 0) {
            isGettingOutOfPenaltyBox = true;

            System.out.println(players.get(currentPlayer) + " is getting out of the penalty box");
            places[currentPlayer] = places[currentPlayer] + roll;
            if (places[currentPlayer] > 11) places[currentPlayer] = places[currentPlayer] - 12;

            System.out.println(players.get(currentPlayer)
                + "'s new location is "
                + places[currentPlayer]);
            System.out.println("The category is " + currentCategory());
            askQuestion();
        }
    }
```

Trivia Game 세부규칙3

기존 코드에서 어느 한 플레이어가 코인 6개를 모르면 게임이 끝난다.

```
do {
    aGame.rolling();

    if (rand.nextInt(9) == 7) {
        notAWinner = aGame.wrongAnswer();
    } else {
        notAWinner = aGame.wasCorrectlyAnswered();
    }
} while (notAWinner);
```

```
public boolean wasCorrectlyAnswered() {
    if (inPenaltyBox[currentPlayer]){
        if (isGettingOutOfPenaltyBox) {
            inPenaltyBox[currentPlayer] = false;
            System.out.println("Answer was correct!!!!");

            purses[currentPlayer]++;
            System.out.println(players.get(currentPlayer) + " now has "
                + purses[currentPlayer] + " Gold Coins.");

            boolean winner = didPlayerWin();
            currentPlayer++;
            if (currentPlayer == players.size()) currentPlayer = 0;

            return winner;
        } else {
            currentPlayer++;
            if (currentPlayer == players.size()) currentPlayer = 0;
            return true;
        }
    } else {
        System.out.println("Answer was correct!!!!");
        purses[currentPlayer]++;
        System.out.println(players.get(currentPlayer) + " now has "
            + purses[currentPlayer] + " Gold Coins.");

        boolean winner = didPlayerWin();
        currentPlayer++;
        if (currentPlayer == players.size()) currentPlayer = 0;

        return winner;
    }
}
```

```
private boolean didPlayerWin() {
    return !(purses[currentPlayer] == 6);
}
```

Chapter2

리팩토링을 위한 Unit Test 준비

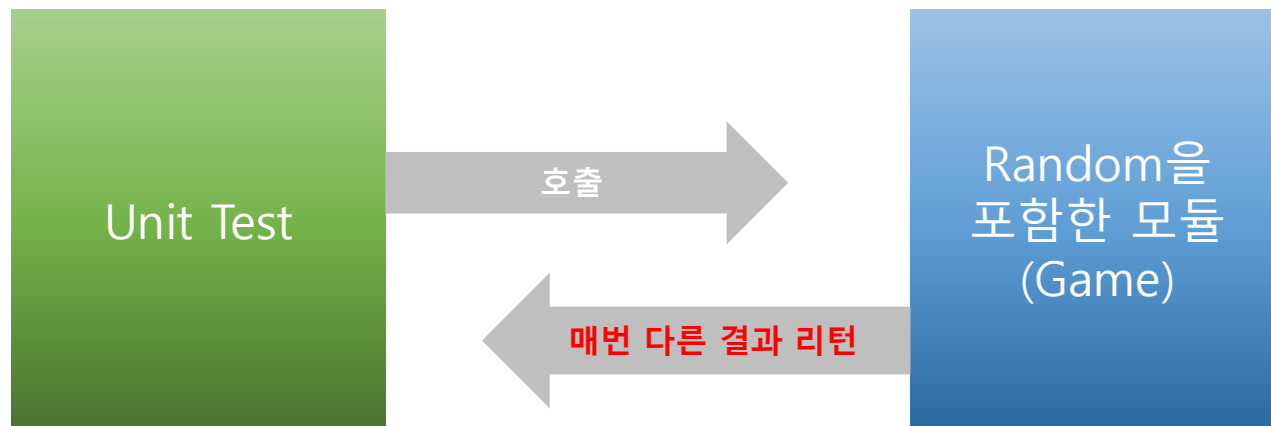
Unit Test가 가능한 구조로 변경 1

- Unit Test 원칙 – Repeatable

테스트는 반복 가능 해야 한다.

→ 테스트 할 때 마다 동일한 입력 값에 대해 출력 값은 일정해야 한다.

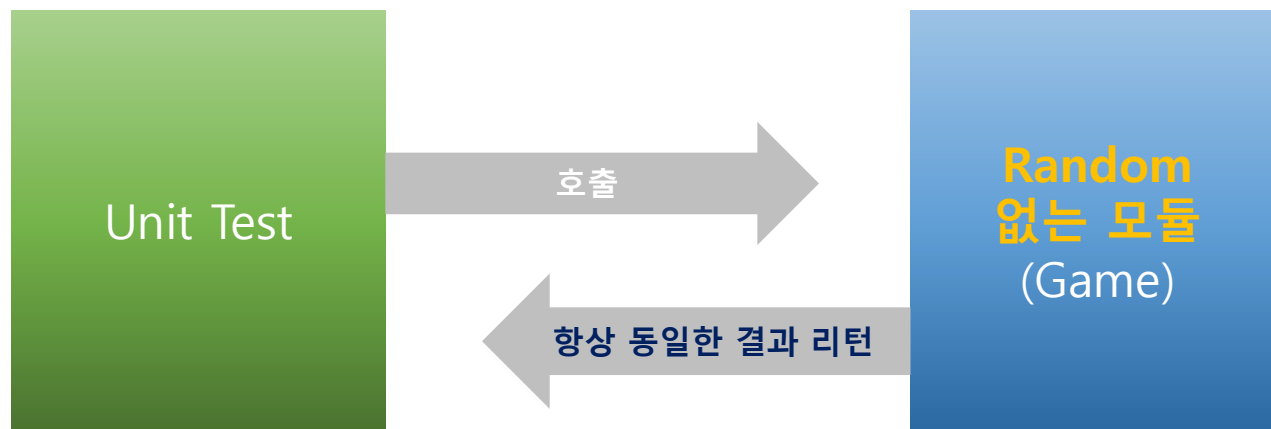
→ 테스트 대상이, 랜덤한 값이 생성되는 경우 UnitTest를 할 수 없다.



현재 Trivia 모듈은 Random을 포함하고 있기에 Repeatable한 Unit Test를 만들 수 없다.

Unit Test가 가능한 구조로 변경 2

Random 생성해주는 코드가 없거나, seed를 넣어주는 코드로 만들어야
UnitTest가 가능한 모듈이 된다.



인자 값으로 주사위 눈금을 넘겨주면
매번 동일한 결과를 얻을 수 있다.

Unit Test가 가능한 구조로 변경 3

- Game 모듈에서 랜덤 값을 구하지 않는다.

```
public void rolling() {  
    Random rand = new Random();  
    int roll = rand.nextInt(6) + 1;  
  
    System.out.println(players.get(currentPla  
    System.out.println("They have rolled a "
```

랜덤 생성 코드 제거

변경

```
public void rolling(int roll) {  
    System.out.println(players.get(  
    System.out.println("They have r
```

roll 파라미터 추가

```
public class GameRunner {  
    public static void main(String[] args) {  
        boolean notAWinner;  
  
        Game aGame = new Game();  
  
        aGame.add("Chet");  
        aGame.add("Pat");  
        aGame.add("Sue");  
  
        Random rand = new Random();  
  
        do {  
            int roll = rand.nextInt(6) + 1;  
            aGame.rolling(roll);  
        }  
    }  
}
```

랜덤 생성 코드 추가

Golden Master 테스트

Golden Master 테스트를 다음과 같은 방식으로 제작한다.

1. Game.java 를 복사하여 **gameBetter.java**를 만든다.
Game.java : 원본
GameBetter.java : 타겟 = 리팩토링 할 대상
2. 원본(Game.java)과 타겟(GameBetter.java)에 똑같은 입력을 넣었을 때,
동일한 출력이 나오는지 비교하는 Unit Test 작성
3. 고정된 Random Seed값 마다 하나의 테스트 케이스를 만든다.
 - 또는 Parameterized testing 수행

[참고] 출력 캡처하기

✓System.out 출력을 캡처하는 소스코드

```
import java.io.ByteArrayOutputStream;
import java.io.PrintStream;

public class Test {
    public static void main(String[] args) {
        // 캡처할 스트림 생성 후, 출력 스트림의 기본값으로 세팅
        ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
        PrintStream originalOut = System.out;
        System.setOut(new PrintStream(outputStream));

        // 출력
        System.out.println("Hello, world!");

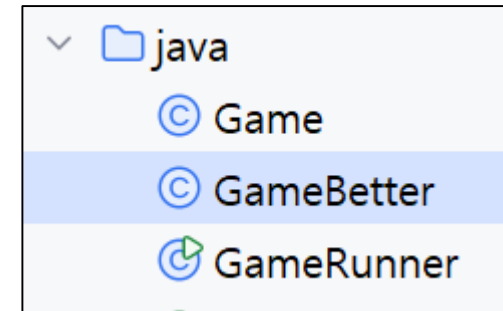
        // 출력 스트림 원상복구
        System.setOut(originalOut);

        // 캡처된 출력 확인
        System.out.println("Captured output: " + outputStream.toString());
    }
}
```

Captured output: Hello, world!

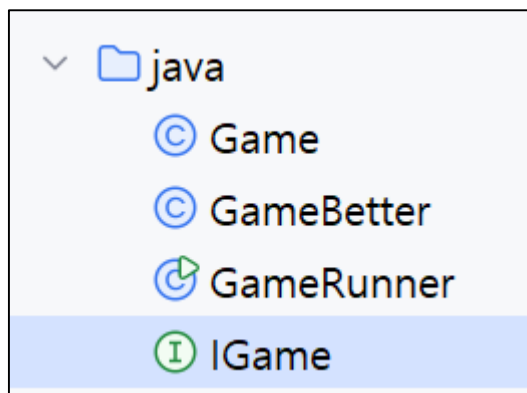
GameBetter.java 파일 생성하기

- ✓ Game.java 파일을 복사하여 GameBetter.java 파일을 생성한다.
- ✓ GameBetter.java 파일은 리팩토링 대상이 된다.



Interface 추가하기

- ✓ Game과 GameBetter 테스트 코드를 만들기 쉽게,
Interface를 하나 추가한다.



```
public interface IGame {  
    public boolean add(String playerName);  
  
    public void rolling(int roll);  
  
    public boolean wasCorrectlyAnswered();  
  
    public boolean wrongAnswer();  
}
```

Test 파일 만들기

✓GameTest.java 파일을 생성하여
우측과 같이 Runner 코드를
가져와서 완성한다.

✓playGame 함수를 호출하여
출력 결과를 비교할 수 있다.

```
class GameTest {  
    void playGame(IGame aGame, int seed) {  
        boolean notAWinner;  
  
        aGame.add("Chet");  
        aGame.add("Pat");  
        aGame.add("Sue");  
  
        Random rand = new Random(seed);  
  
        do {  
            int roll = rand.nextInt(6) + 1;  
            aGame.rolling(roll);  
  
            if (rand.nextInt(9) == 7) {  
                notAWinner = aGame.wrongAnswer();  
            } else {  
                notAWinner = aGame.wasCorrectlyAnswered();  
            }  
        } while (notAWinner);  
    }  
}
```

Test Case 추가하기

✓playGame을 두 번 호출하고,
두 결과값을 비교하는 방식으로
검증한다.

✓여기까지 완성된 소스코드

- <https://gist.github.com/mincoding1/402d03b655a7aa0946f2f8716daf56cf>

```
@Test
void goldenMaster() {
    //arrange
    ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
    PrintStream originalOut = System.out;
    System.setOut(new PrintStream(outputStream));

    int randomSeed = 1;
    playGame(new Game(), randomSeed);
    String expected = outputStream.toString();
    outputStream.reset(); //출력 스트림 리셋

    //actual
    playGame(new GameBetter(), randomSeed);
    String actual = outputStream.toString();
    System.setOut(originalOut); //출력 스트림 원상복구

    //assert
    assertEquals(expected, actual);
}
```

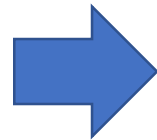
```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
```

개선하기 : Parameterized Test

- ✓ JUnit의 Parameterized Test 기능을 사용하여,
파라미터 값을 변경하며 테스트하는 코드로 Unit Test를 개선한다.
- Import는 alt + Enter로 해결하자.

```
@Test
void goldenMaster() {
    //arrange
    ByteArrayOutputStream outputStream = new
    PrintStream originalOut = System.out;
    System.setOut(new PrintStream(outputStream));

    int randomSeed = 1;
    playGame(new Game(), randomSeed);
    String expected = outputStream.toString();
}
```



```
@ParameterizedTest
@ValueSource(ints = {1, 50, 100, 777})
void goldenMaster(int randomSeed) {
    //arrange
    ByteArrayOutputStream outputStream = new
    PrintStream originalOut = System.out;
    System.setOut(new PrintStream(outputStream));

    playGame(new Game(), randomSeed);
    String expected = outputStream.toString();
}
```

✓ GameTest

✓ goldenMaster(int)

✓ [1] 1

✓ [2] 50

✓ [3] 100

✓ [4] 777

Chapter3

리팩토링 시작

연습 포인트

- 중복 코드 식별 후 Extract Method
- Pure function 으로 추출하기
- Game 클래스로부터 다른 클래스 추출하기

객체지향의 원칙 적용하기

- 책임 식별하기 (SRP, Single Responsibility Principle)
- 중복 제거 (DRY, Do not Repeat Yourself)

[추가 미션] 리팩토링 이후, 기능추가 요구사항

기능추가 요구사항

1. 최대 플레이어 수를 6명으로
2. 새로운 질문 카테고리 추가 ex) "ART"
3. 게임을 시작하려면 최소 2명 이상
4. 게임이 시작된 이후 새로운 플레이어 추가 금지
5. 플레이어 동일한 이름 금지
6. 3번 연속 정답을 맞추는 경우, 그 다음 정답에 대해 2점을 얻기